

LAB

Topic 3: Complex SQL Queries



Lecturer: Lê Thị Nhàn, PhD.
Lab instructor: Nguyễn Ngọc Toàn, MSc.
Nguyễn Ngọc Minh Châu, MSc.

1) Common Table Expression – CTE

Syntax

```
WITH cte_name (column1, column2, ...) AS (  
    SELECT ...  
    FROM ...  
    WHERE ...  
)  
SELECT *  
FROM cte_name;
```

Note:

- cte_name: A user-defined name for the CTE, which should be unique within the SQL statement.
- (column1, column2, ...): **Optional**. A list of column names for the CTE to provide names for the columns within the CTE.
- **(Important):** In this course, you are not allowed to use CTE queries unless the questions permit you to use them.

Non-recursive CTE

Example: List Students with Their Average Grade

- With CTE

```
WITH StudentAverage AS (  
    SELECT  
        student_id,  
        AVG(grade_100) AS avg_grade  
    FROM GradeReport  
    GROUP BY student_id  
)  
SELECT  
    s.student_id,  
    s.student_name,  
    sa.avg_grade  
FROM Student s  
JOIN StudentAverage sa ON s.student_id = sa.student_id  
ORDER BY sa.avg_grade DESC;
```

- Without CTE:

```
SELECT
    s.student_id,
    s.student_name,
    AVG(gr.grade_100) AS avg_grade
FROM Student s
JOIN GradeReport gr ON s.student_id = gr.student_id
GROUP BY s.student_id, s.student_name
ORDER BY avg_grade DESC;
```

Recursive CTE

Note: Recursive CTEs consist of **two** parts:

- **Anchor member:** The initial query that selects the base case.
- **Recursive member:** The query that references the CTE itself.

Example: Find all direct and indirect prerequisites for a given course (e.g, 'CS10')

- With recursive CTE

```
-- WITH RECURSIVE ... AS for some old versions of SSMS
WITH PrerequisiteChain AS (
    -- Anchor member: Direct prerequisites
    SELECT
        course_id,
        prerequisite_id
    FROM Prerequisite
    WHERE course_id = 'CS10'
    UNION ALL
    -- Recursive member: Get prerequisites of prerequisites
    SELECT
        p.course_id,
        p.prerequisite_id
    FROM Prerequisite p
    INNER JOIN PrerequisiteChain pc ON p.course_id = pc.prerequisite_id
)
SELECT DISTINCT
```

```
c.course_id AS target_course,  
c.course_name AS target_course_name,  
pc.prerequisite_id,  
cp.course_name AS prerequisite_name  
FROM PrerequisiteChain pc  
JOIN Course c ON c.course_id = 'CS10'  
JOIN Course cp ON cp.course_id = pc.prerequisite_id;
```

- Without CTE (3 loops):

```
SELECT  
  c.course_id AS target_course,  
  c.course_name AS target_course_name,  
  p1.prerequisite_id AS level_1_id,  
  c1.course_name AS level_1_name,  
  p2.prerequisite_id AS level_2_id,  
  c2.course_name AS level_2_name,  
  p3.prerequisite_id AS level_3_id,  
  c3.course_name AS level_3_name  
FROM Course c  
LEFT JOIN Prerequisite p1 ON c.course_id = p1.course_id  
LEFT JOIN Course c1 ON p1.prerequisite_id = c1.course_id  
LEFT JOIN Prerequisite p2 ON p1.prerequisite_id = p2.course_id  
LEFT JOIN Course c2 ON p2.prerequisite_id = c2.course_id  
LEFT JOIN Prerequisite p3 ON p2.prerequisite_id = p3.course_id  
LEFT JOIN Course c3 ON p3.prerequisite_id = c3.course_id  
WHERE c.course_id = 'CS10';
```

2) Conditional Data Manipulation in SQL

INSERT INTO New Table from Existing Table (with Condition)

```
INSERT INTO target_table(column1, column2,...)
SELECT column1, column2,...
FROM source_table
WHERE ...
```

Notes:

- The number, order, and data types of columns in **INSERT INTO (...)** must exactly match those in the **SELECT** clause

EXAMPLE

Create a new table named HonorStudents to store the list of students with average grades over 85.

```
-- Step 1: Create a new table to store high-performing students.
CREATE TABLE HonorStudents (
    student_id VARCHAR(9) PRIMARY KEY,
    student_name NVARCHAR(50),
    department_id VARCHAR(5),
    avg_grade FLOAT
)

-- Step 2: Insert students who have an average grade over 85 into the new table.
INSERT INTO HonorStudents (student_id, student_name, department_id, avg_grade)
SELECT
    S.student_id,
    S.student_name,
    S.department_id,
    AVG(G.grade_100) as avg_grade
FROM Student S
JOIN GradeReport G ON S.student_id = G.student_id
GROUP BY S.student_id, S.student_name, S.department_id
HAVING AVG(G.grade_100) > 85
```

DELETE (with Condition)

Query: Delete all students who have no department on GradeReport.

Using EXISTS

```
DELETE
FROM GradeReport
WHERE EXISTS (
  SELECT *
  FROM Student s
  WHERE s.student_id = GradeReport.student_id AND s.department_ID IS NULL
)
```

Using IN

```
DELETE
FROM GradeReport
WHERE Student_ID IN (
  SELECT Student_ID
  FROM Student
  WHERE department_ID IS NULL
)
```

Using WITH and IN

```
WITH StudentNullDepartment AS(
  SELECT student_id
  FROM Student
  WHERE department_id IS NULL
)
DELETE
FROM GradeReport
WHERE Student_ID IN (select * FROM StudentNullDepartment)
```

UPDATE (with Condition)

Query: Increase the salary of all instructors who belong to the 'Software Engineering' department by 10%.

```
UPDATE I
```

```
SET I.salary = I.salary * 1.1
FROM Instructor I
JOIN Department D ON I.department_id = D.department_id
WHERE D.department_name = 'Software Engineering'
```

- Without CTE (3 loops):

```
SELECT
  c.course_id AS target_course,
  c.course_name AS target_course_name,
  p1.prerequisite_id AS level_1_id,
  c1.course_name AS level_1_name,
  p2.prerequisite_id AS level_2_id,
  c2.course_name AS level_2_name,
  p3.prerequisite_id AS level_3_id,
  c3.course_name AS level_3_name
FROM Course c
LEFT JOIN Prerequisite p1 ON c.course_id = p1.course_id
LEFT JOIN Course c1 ON p1.prerequisite_id = c1.course_id
LEFT JOIN Prerequisite p2 ON p1.prerequisite_id = p2.course_id
LEFT JOIN Course c2 ON p2.prerequisite_id = c2.course_id
LEFT JOIN Prerequisite p3 ON p2.prerequisite_id = p3.course_id
LEFT JOIN Course c3 ON p3.prerequisite_id = c3.course_id
WHERE c.course_id = 'CS10';
```